

100 Python & Bash Scripting Codes and its Solutions

Problem 1 — Top 3 client IPs from an NGINX access log

Problem: Given an nginx access-log as a string, find the top 3 client IPs by request count.

```
1 # solution: top3_ips.py
2 from collections import Counter
3 import re
4
5 nginx_log = """
6 192.0.2.1 - - [06/Dec/2025:12:00:00 +0530] "GET /index.html HTTP/1.1"
7 200 612
8 203.0.113.5 - - [06/Dec/2025:12:00:01 +0530] "POST /api/v1/login
9 HTTP/1.1" 302 245
10 192.0.2.1 - - [06/Dec/2025:12:00:02 +0530] "GET /style.css HTTP/1.1"
11 200 1024
12 198.51.100.7 - - [06/Dec/2025:12:00:03 +0530] "GET /api/v1/data
13 HTTP/1.1" 200 2048
14 203.0.113.5 - - [06/Dec/2025:12:00:04 +0530] "GET /dashboard HTTP/1.1"
15 200 512
16 192.0.2.1 - - [06/Dec/2025:12:00:05 +0530] "GET /favicon.ico HTTP/1.1"
17 200 150
18 """
19
20 ip_re = re.compile(r'^(\d+\.\d+\.\d+\.\d+)')
21 ips = ip_re.findall(nginx_log)
22 counts = Counter(ips)
23 top3 = counts.most_common(3)
24 print("Top 3 IPs:")
25 for ip, c in top3:
26     print(f"{ip} - {c} requests")
27
```

Expected output

```
1 Top 3 IPs:
2 192.0.2.1 - 3 requests
3 203.0.113.5 - 2 requests
4 198.51.100.7 - 1 requests
5
```

Problem 2 — Compute network address and broadcast from IP + mask

Problem: Given `ip = "10.55.252.10"` and `mask = "255.255.255.224"`, compute the network address, CIDR (e.g. `/27`), first usable, last usable, and broadcast.

```
1 # solution: cidr_calc.py
2 import ipaddress
3
4 ip = "10.55.252.10"
5 mask = "255.255.255.224"
6
7 # Create IPv4Network using ip and netmask
8 network = ipaddress.IPv4Network(f"{ip}/{mask}", strict=False)
9
10 network_address = str(network.network_address)
11 prefixlen = network.prefixlen
12 broadcast = str(network.broadcast_address)
13
```

```

14 # compute first and last usable (exclude network and broadcast if
    possible)
15 usable = list(network.hosts())
16 first_usable = str(usable[0]) if usable else None
17 last_usable = str(usable[-1]) if usable else None
18
19 print(f"Network: {network_address}/{prefixlen}")
20 print(f"Broadcast: {broadcast}")
21 print(f"First usable: {first_usable}")
22 print(f"Last usable: {last_usable}")
23

```

Expected output

```

1 Network: 10.55.252.0/27
2 Broadcast: 10.55.252.31
3 First usable: 10.55.252.1
4 Last usable: 10.55.252.30
5

```

Problem 3 — Simple `tail -n` and `tail -f` simulator (generator)

Problem: Implement `tail_n` (last N lines) and a `tail_f` generator that yields appended lines (simulate appending by feeding new lines via a list).

```

1 # solution: tail_sim.py
2 from collections import deque
3 import time
4 from typing import Generator, Iterable
5
6 def tail_n(lines: Iterable[str], n: int):
7     dq = deque(maxlen=n)
8     for line in lines:
9         dq.append(line.rstrip("\n"))
10    return list(dq)
11
12 def tail_f(initial_lines: Iterable[str], appended_lines:
    Iterable[str]) -> Generator[str, None, None]:
13     # yield initial content
14     for l in initial_lines:
15         yield l.rstrip("\n")
16     # then simulate appended lines arriving later
17     for l in appended_lines:
18         time.sleep(0.01) # short sleep to simulate time gap
19         yield l.rstrip("\n")
20
21 # demo
22 initial = ["a\n", "b\n", "c\n"]
23 appended = ["d\n", "e\n"]
24 print("tail -n 2:", tail_n(initial, 2))
25 print("tail -f outputs:")
26 for ln in tail_f(initial, appended):
27     print(ln)
28

```

Expected output

```

1 tail -n 2: ['b', 'c']
2 tail -f outputs:
3 a
4 b
5 c
6 d
7 e
8

```

Note: In real `tail -f` you'd block on file changes — here we simulate.

Problem 4 — Parallel file hash checks with retries (ThreadPoolExecutor + retry)

Problem: Given a list of "files" (strings representing content), compute SHA256 hashes in parallel and retry a failing job up to 2 times.

```
1 # solution: parallel_hash_retry.py
2 import hashlib
3 from concurrent.futures import ThreadPoolExecutor, as_completed
4 import random
5 import time
6
7 files = {
8     "file1": "content-one",
9     "file2": "content-two",
10    "file3": "content-three",
11 }
12
13 def compute_hash(name, content, fail_rate=0.3):
14     # simulate transient failure
15     if random.random() < fail_rate:
16         raise RuntimeError("transient error")
17     h = hashlib.sha256(content.encode()).hexdigest()
18     return name, h
19
20 def compute_with_retries(name, content, retries=2):
21     attempt = 0
22     while True:
23         try:
24             return compute_hash(name, content)
25         except Exception as e:
26             attempt += 1
27             if attempt > retries:
28                 return name, f"FAILED after {retries} retries: {e}"
29             time.sleep(0.01) # backoff
30
31 results = {}
32 with ThreadPoolExecutor(max_workers=3) as ex:
33     futures = {ex.submit(compute_with_retries, n, c): n for n, c in
34                files.items()}
35     for fut in as_completed(futures):
36         name, res = fut.result()
37         results[name] = res
38
39 for k in sorted(results):
40     print(k, results[k])
```

Example expected output (note: nondeterministic due to random failures; typical run:)

```
1 file1 4a1f... (sha256 hex)
2 file2 FAILED after 2 retries: transient error
3 file3 e2b7... (sha256 hex)
4
```

(In actual runs, some may fail or succeed depending on randomness.)

Problem 5 — YAML validator for minimal Kubernetes Deployment

Problem: Given YAML text, validate it contains `apiVersion`, `kind: Deployment`, `metadata.name`, and `spec.template.spec.containers` with at least one container.

```

1 # solution: k8s_validate.py
2 # Requires pyyaml: pip install pyyaml
3 import yaml
4
5 def validate_deployment(yaml_text: str):
6     try:
7         obj = yaml.safe_load(yaml_text)
8     except Exception as e:
9         return False, f"YAML parse error: {e}"
10
11     if not isinstance(obj, dict):
12         return False, "Top-level YAML is not a mapping"
13
14     if obj.get("kind") != "Deployment":
15         return False, "kind is not Deployment"
16
17     if "apiVersion" not in obj:
18         return False, "apiVersion missing"
19     if "metadata" not in obj or "name" not in obj["metadata"]:
20         return False, "metadata.name missing"
21     spec = obj.get("spec", {})
22     tpl = spec.get("template", {})
23     pod_spec = tpl.get("spec", {})
24     containers = pod_spec.get("containers")
25     if not containers or not isinstance(containers, list):
26         return False, "containers missing or invalid"
27     if len(containers) == 0:
28         return False, "no containers defined"
29     return True, "Valid Deployment"
30
31 # demo YAML
32 demo = """
33 apiVersion: apps/v1
34 kind: Deployment
35 metadata:
36   name: my-app
37 spec:
38   template:
39     spec:
40       containers:
41         - name: app
42           image: nginx:1.25
43 """
44
45 print(validate_deployment(demo))
46

```

Expected output

```

1 (True, 'Valid Deployment')
2

```

Problem 6 — Log rotation: keep latest N backups

Problem: Implement a function `rotate_log(path, n)` which renames `path` → `path.1`, `path.1` → `path.2`, ..., dropping the oldest, and create a new empty `path`.

```

1 # solution: rotate_logs.py
2 import os
3 import shutil
4
5 def rotate_log(path, n=5):
6     if n < 1:
7         raise ValueError("n must be >=1")
8     # remove oldest
9     oldest = f"{path}.{n}"
10    if os.path.exists(oldest):

```

```

11     os.remove(oldest)
12     # shift down
13     for i in range(n-1, 0, -1):
14         src = f"{path}.{i}"
15         dst = f"{path}.{i+1}"
16         if os.path.exists(src):
17             os.rename(src, dst)
18     # current -> .1
19     if os.path.exists(path):
20         os.rename(path, f"{path}.1")
21     # create new empty log
22     open(path, "w").close()
23
24 # demo using temp files
25 demo_path = "app.log"
26 # create initial files
27 for i in range(1,4):
28     with open(f"{demo_path}.{i}", "w") as f:
29         f.write(f"old{i}\n")
30 with open(demo_path, "w") as f:
31     f.write("current\n")
32
33 rotate_log(demo_path, n=4)
34 print("Files after rotate:", sorted([f for f in os.listdir(".") if
35     f.startswith("app.log")]))
36 # cleanup
37 for f in [p for p in os.listdir(".") if p.startswith("app.log")]:
38     os.remove(f)

```

Expected output

```

1 Files after rotate: ['app.log', 'app.log.1', 'app.log.2', 'app.log.3',
2 'app.log.4']

```

Problem 7 — Exponential backoff decorator

Problem: Write a decorator `@with_backoff(retries=3)` that retries a function with exponential backoff (1s, 2s, 4s... — make sleeps configurable). Return last exception if all retries fail.

```

1 # solution: backoff_decorator.py
2 import time
3 import functools
4
5 def with_backoff(retries=3, base=1.0):
6     def deco(func):
7         @functools.wraps(func)
8         def wrapper(*args, **kwargs):
9             attempt = 0
10            while True:
11                try:
12                    return func(*args, **kwargs)
13                except Exception as e:
14                    attempt += 1
15                    if attempt > retries:
16                        raise
17                    sleep = base * (2 ** (attempt - 1))
18                    time.sleep(0.01) # use short sleep in demo;
19                    # replace with time.sleep(sleep) in real code
20            return wrapper
21        return deco
22
23 # demo
24 counter = {"calls": 0}
25 @with_backoff(retries=2)
26 def flaky():

```

```

26     counter["calls"] += 1
27     if counter["calls"] < 3:
28         raise RuntimeError("fail")
29     return "ok"
30
31 print(flaky()) # will succeed on 3rd call
32

```

Expected output

```

1 ok
2

```

Problem 8 — Small CLI: parse subcommands `deploy` and `destroy`

Problem: Implement a CLI that supports `deploy --env ENV` and `destroy --env ENV`, using `argparse`.

```

1 # solution: simple_cli.py
2 import argparse
3
4 def deploy(env):
5     print(f"Deploying to {env}")
6
7 def destroy(env):
8     print(f"Destroying resources in {env}")
9
10 def main(argv=None):
11     parser = argparse.ArgumentParser(prog="infra")
12     sub = parser.add_subparsers(dest="cmd", required=True)
13
14     p_deploy = sub.add_parser("deploy")
15     p_deploy.add_argument("--env", required=True)
16
17     p_destroy = sub.add_parser("destroy")
18     p_destroy.add_argument("--env", required=True)
19
20     args = parser.parse_args(argv)
21     if args.cmd == "deploy":
22         deploy(args.env)
23     elif args.cmd == "destroy":
24         destroy(args.env)
25
26 if __name__ == "__main__":
27     # demo simulated calls:
28     main(["deploy", "--env", "tst"])
29     main(["destroy", "--env", "prd"])
30

```

Expected output

```

1 Deploying to tst
2 Destroying resources in prd
3

```

Problem 9 — Compute SHA256 fingerprint of an SSH public key string

Problem: Given an SSH public key (one-line `ssh-rsa AAAAB3... comment`), compute its SHA256 fingerprint (Base64, like `ssh-keygen -lf`).

```

1 # solution: ssh_fp.py
2 import base64
3 import hashlib

```

```

4
5 def ssh_sha256_fingerprint(pubkey_line: str):
6     # format: <type> <base64> [comment]
7     parts = pubkey_line.strip().split()
8     if len(parts) < 2:
9         raise ValueError("invalid public key line")
10    b64 = parts[1]
11    key_bytes = base64.b64decode(b64)
12    sha256 = hashlib.sha256(key_bytes).digest()
13    fp_b64 = base64.b64encode(sha256).decode().rstrip("=")
14    return f"SHA256:{fp_b64}"
15
16 # demo using a short fake key (not a real rsa key, but for demo)
17 demo_key = "ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQCFakekeycomment"
18 print(ssh_sha256_fingerprint(demo_key))
19

```

Expected output (example; will vary by key)

```

1 SHA256:2nK8... (base64 without padding)
2

```

Problem 10 — Simple presigned-like token generator for an artifact store

Problem: Implement `generate_presigned(name, expiry_seconds=3600)` that returns a URL-like token containing HMAC-SHA256 signature and expiry timestamp. Validate function `validate_token`.

```

1 # solution: presigned_sim.py
2 import hmac
3 import hashlib
4 import time
5 import urllib.parse
6
7 SECRET = b"super-secret-key"
8
9 def generate_presigned(name, expiry_seconds=3600):
10     expires = int(time.time()) + expiry_seconds
11     data = f"{name}:{expires}".encode()
12     sig = hmac.new(SECRET, data, hashlib.sha256).hexdigest()
13     token = urllib.parse.quote_plus(f"{name}:{expires}:{sig}")
14     return f"https://artifacts.example/download?token={token}"
15
16 def validate_token(token_qs):
17     token = urllib.parse.unquote_plus(token_qs)
18     try:
19         name, expires_str, sig = token.split(":")
20     except ValueError:
21         return False, "invalid format"
22     expires = int(expires_str)
23     if time.time() > expires:
24         return False, "expired"
25     expected = hmac.new(SECRET, f"{name}:{expires}".encode(),
26                          hashlib.sha256).hexdigest()
27     if not hmac.compare_digest(expected, sig):
28         return False, "invalid signature"
29     return True, name
30
31 # demo
32 url = generate_presigned("release-1.2.3.tar.gz", expiry_seconds=60)
33 print("URL:", url)
34 qs = url.split("token=")[1]
35 print("Validate:", validate_token(qs))
36

```

Expected output

```
1 URL: https://artifacts.example/download?token=release-
  1.2.3.tar.gz:170...:9f7a...
2 Validate: (True, 'release-1.2.3.tar.gz')
3
```

Problem 11 — Parse a JSON file and extract a field

Python Solution

```
1 import json
2
3 data = '{"app":"nginx","version":"1.25.3","status":"running"}'
4 obj = json.loads(data)
5 print(obj["version"])
6
```

Shell Script Solution

```
1 #!/bin/bash
2 data='{"app":"nginx","version":"1.25.3","status":"running"}'
3 echo "$data" | jq -r '.version'
4
```

Expected Output

```
1 1.25.3
2
```

Problem 12 — Count number of lines in a log file containing the word "ERROR"

Python Solution

```
1 log = """INFO boot complete
2 ERROR failed to start service
3 WARN retry attempt 1
4 ERROR connection timeout"""
5
6 count = sum(1 for line in log.splitlines() if "ERROR" in line)
7 print(count)
8
```

Shell Script Solution

```
1 #!/bin/bash
2 log="INFO boot complete
3 ERROR failed to start service
4 WARN retry attempt 1
5 ERROR connection timeout"
6
7 echo "$log" | grep -c "ERROR"
8
```

Expected Output

```
1 2
2
```

Problem 13 — Check if a port is open on a remote server

Python Solution

```
1 import socket
```



```

2
3 host = "google.com"
4 port = 443
5
6 s = socket.socket()
7 try:
8     s.settimeout(2)
9     s.connect((host, port))
10    print("open")
11 except:
12    print("closed")
13 finally:
14    s.close()
15

```

Shell Script Solution

```

1 #!/bin/bash
2 host="google.com"
3 port=443
4
5 timeout 2 bash -c "</dev/tcp/$host/$port" && echo "open" || echo
  "closed"
6

```

Expected Output

```

1 open
2

```

Problem 14 — Convert a list of filenames to lowercase

Python Solution

```

1 files = ["LOG.TXT", "App.CONF", "Setup.SH"]
2 lowered = [f.lower() for f in files]
3 print(lowered)
4

```

Shell Script Solution

```

1 #!/bin/bash
2 files=("LOG.TXT" "App.CONF" "Setup.SH")
3
4 for f in "${files[@]"; do
5     echo "${f,,}"
6 done
7

```

Expected Output

Python:

```

1 ['log.txt', 'app.conf', 'setup.sh']
2

```

Shell:

```

1 log.txt
2 app.conf
3 setup.sh
4

```

Problem 15 — Find largest file in a directory

Python Solution

```
1 import os
2
3 dir_path = "."
4 largest = max(
5     (f for f in os.listdir(dir_path) if os.path.isfile(f)),
6     key=os.path.getsize
7 )
8 print(largest)
9
```

Shell Script Solution

```
1 #!/bin/bash
2 ls -S | head -1
3
```

Expected Output

(Depends on files present, example)

```
1 config.log
2
```

Problem 16 — Remove duplicate lines from a file (retain order)

Python Solution

```
1 log = ""a
2 b
3 a
4 c
5 b""
6
7 seen = set()
8 for line in log.splitlines():
9     if line not in seen:
10         print(line)
11         seen.add(line)
12
```

Shell Script Solution

```
1 #!/bin/bash
2 echo -e "a\nb\na\nc\nb" | awk '!seen[$0]++'
3
```

Expected Output

```
1 a
2 b
3 c
4
```

Problem 17 — Check disk usage above threshold

Python Solution

```
1 import shutil
2
3 total, used, free = shutil.disk_usage("/")
```

```
4 usage = used / total * 100
5
6 print("High usage" if usage > 80 else "OK")
7
```

Shell Script Solution

```
1 #!/bin/bash
2 usage=$(df / | awk 'NR==2 {print $5}' | tr -d '%')
3
4 if [ $usage -gt 80 ]; then
5     echo "High usage"
6 else
7     echo "OK"
8 fi
9
```

Expected Output

(Example)

```
1 OK
2
```

Problem 18 — Extract unique IPs from a log

Python Solution

```
1 import re
2
3 log = """
4 10.1.1.1 GET /
5 10.1.1.2 POST /api
6 10.1.1.1 GET /home
7 """
8
9 ips = sorted(set(re.findall(r'\d+\.\d+\.\d+\.\d+', log)))
10 print(ips)
11
```

Shell Script Solution

```
1 #!/bin/bash
2 echo -e "10.1.1.1 GET /\n10.1.1.2 POST /api\n10.1.1.1 GET /home" \
3 | grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' | sort -u
4
```

Expected Output

```
1 ['10.1.1.1', '10.1.1.2']
2
```

Shell:

```
1 10.1.1.1
2 10.1.1.2
3
```

Problem 19 — Send an alert email if a service is down

Python Solution

(Mocked without SMTP)

```
1 service_running = False
2
3 if not service_running:
4     print("Sending email: service is DOWN")
5 else:
6     print("Service healthy")
7
```

Shell Script Solution

```
1 #!/bin/bash
2 service_running=0
3
4 if [ $service_running -eq 0 ]; then
5     echo "Sending email: service is DOWN"
6 else
7     echo "Service healthy"
8 fi
9
```

Expected Output

```
1 Sending email: service is DOWN
2
```

Problem 20 — Convert epoch timestamp to human-readable

Python Solution

```
1 import datetime
2
3 epoch = 1700000000
4 print(datetime.datetime.fromtimestamp(epoch))
5
```

Shell Script Solution

```
1 #!/bin/bash
2 epoch=1700000000
3 date -d "@$epoch"
4
```

Expected Output

```
1 2023-11-14 00:53:20
2
```

Great — here is **Batch 3 (Problems 21–30)** with:

- ✓ Python Solution
- ✓ Shell Script Solution
- ✓ Expected Output

If you want **Batch 4**, just say “**next**”.

Problem 21 — Check if a package is installed (Linux)

Python Solution

```
1 import subprocess
2
```

```
3 pkg = "curl"
4 result = subprocess.run(["dpkg", "-l", pkg], stdout=subprocess.PIPE,
5 stderr=subprocess.PIPE)
6 print("installed" if result.returncode == 0 else "not installed")
7
```

Shell Script Solution

```
1 #!/bin/bash
2 pkg="curl"
3
4 dpkg -l "$pkg" &>/dev/null && echo "installed" || echo "not installed"
5
```

Expected Output

```
1 installed
2
```

Problem 22 — Sort a CSV file by second column

Python Solution

```
1 data = """name,age
2 john,34
3 alice,29
4 mark,40
5 """
6
7 lines = data.splitlines()
8 header = lines[0]
9 records = [line.split(",") for line in lines[1:]]
10
11 sorted_data = sorted(records, key=lambda x: int(x[1]))
12 for r in sorted_data:
13     print(",".join(r))
14
```

Shell Script Solution

```
1 #!/bin/bash
2 echo -e "name,age\njohn,34\nalice,29\nmark,40" | sort -t"," -k2 -n
3
```

Expected Output

```
1 alice,29
2 john,34
3 mark,40
4
```

Problem 23 — Remove files older than X days

Python Solution

```
1 import os, time
2
3 days = 7
4 threshold = time.time() - days * 86400
5
6 # demo: iterate over current directory
7 for f in os.listdir("."):
8     if os.path.isfile(f) and os.path.getmtime(f) < threshold:
9         print(f"Old file: {f}")
```

```
10
```

Shell Script Solution

```
1 #!/bin/bash
2 find . -type f -mtime +7 -print
3
```

Expected Output

(Example)

```
1 Old file: old.log
2
```

Problem 24 — Generate a random password

Python Solution

```
1 import secrets
2 import string
3
4 chars = string.ascii_letters + string.digits
5 password = "".join(secrets.choice(chars) for _ in range(12))
6 print(password)
7
```

Shell Script Solution

```
1 #!/bin/bash
2 tr -dc A-Za-z0-9 </dev/urandom | head -c 12
3 echo
4
```

Expected Output

```
1 xF92asJ1dkP0
2
```

(Will vary)

Problem 25 — Find the PID of a running process

Python Solution

```
1 import subprocess
2
3 process = "ssh"
4 output = subprocess.getoutput(f"pgrep -f {process}")
5 print(output if output else "not running")
6
```

Shell Script Solution

```
1 #!/bin/bash
2 process="ssh"
3 pgrep -f "$process" || echo "not running"
4
```

Expected Output

```
1 1234
2
```

(Example PID)

Problem 26 — Convert text to uppercase

Python Solution

```
1 text = "Welcome to DevOps"
2 print(text.upper())
3
```

Shell Script Solution

```
1 #!/bin/bash
2 echo "Welcome to DevOps" | tr '[:lower:]' '[:upper:]'
3
```

Expected Output

```
1 WELCOME TO DEVOPS
2
```

Problem 27 — Validate a domain name

Python Solution

```
1 import re
2
3 domain = "example.com"
4 pattern = r"^(?!-)[A-Za-z0-9-]{1,63}(?!-)\.[A-Za-z]{2,6}$"
5 print("valid" if re.match(pattern, domain) else "invalid")
6
```

Shell Script Solution

```
1 #!/bin/bash
2 domain="example.com"
3
4 if [[ $domain =~ ^[A-Za-z0-9.-]+$ ]]; then
5     echo "valid"
6 else
7     echo "invalid"
8 fi
9
```

Expected Output

```
1 valid
2
```

Problem 28 — Print directory tree structure

Python Solution

```
1 import os
2
3 def tree(dir, prefix=""):
4     for item in os.listdir(dir):
5         path = os.path.join(dir, item)
6         print(prefix + "|-- " + item)
7         if os.path.isdir(path):
8             tree(path, prefix + "    ")
9
```

```
10 tree(".")
11
```

Shell Script Solution

```
1 #!/bin/bash
2 tree .
3
```

Expected Output

(Example)

```
1 |-- app.py
2 |-- config
3     |-- dev.conf
4     |-- prod.conf
5
```

Problem 29 — Test internet connectivity

Python Solution

```
1 import subprocess
2
3 result = subprocess.run(["ping", "-c", "1", "8.8.8.8"],
4                          stdout=subprocess.PIPE)
5 print("online" if result.returncode == 0 else "offline")
```

Shell Script Solution

```
1 #!/bin/bash
2 ping -c1 8.8.8.8 &>/dev/null && echo "online" || echo "offline"
3
```

Expected Output

```
1 online
2
```

Problem 30 — Compress a directory to a .tar.gz file

Python Solution

```
1 import tarfile
2
3 with tarfile.open("backup.tar.gz", "w:gz") as tar:
4     tar.add("myfolder")
5 print("compressed")
6
```

Shell Script Solution

```
1 #!/bin/bash
2 tar -czf backup.tar.gz myfolder
3 echo "compressed"
4
```

Expected Output

```
1 compressed
2
```

31. Count number of files per extension in a directory

Python Solution

```
1 import os
2 from collections import Counter
3
4 def count_extensions(path):
5     files = [f.split('.')[-1] for f in os.listdir(path) if
6 os.path.isfile(os.path.join(path, f))]
7     return Counter(files)
8
9 print(count_extensions("/tmp"))
```

Python Output (example)

```
1 Counter({'log': 5, 'txt': 2, 'conf': 1})
2
```

Shell Script Solution

```
1 #!/bin/bash
2 path="/tmp"
3 declare -A count
4
5 for f in "$path"/*; do
6     if [[ -f "$f" ]]; then
7         ext="${f##*.}"
8         ((count[$ext]++))
9     fi
10 done
11
12 for ext in "${!count[@]}"; do
13     echo "$ext : ${count[$ext]}"
14 done
15
```

Shell Output (example)

```
1 log : 5
2 txt : 2
3 conf : 1
4
```

32. Check if a service is running and restart if stopped

Python Solution

```
1 import subprocess
2
3 service = "nginx"
4 status = subprocess.run(["systemctl", "is-active", service],
5 capture_output=True, text=True)
6
7 if status.stdout.strip() != "active":
8     print(f"{service} is down. Restarting...")
9     subprocess.run(["sudo", "systemctl", "restart", service])
10 else:
11     print(f"{service} is running")
```

Python Output

```
1 nginx is running
2
```

Shell Script Solution

```
1 service=nginx
2 if ! systemctl is-active --quiet "$service"; then
3     echo "$service is down. Restarting..."
4     sudo systemctl restart "$service"
5 else
6     echo "$service is running"
7 fi
8
```

Shell Output

```
1 nginx is running
2
```

33. Validate if a YAML file is valid

Python Solution

```
1 import yaml
2
3 try:
4     with open("config.yaml") as f:
5         yaml.safe_load(f)
6         print("Valid YAML")
7 except yaml.YAMLError as e:
8     print("Invalid YAML:", e)
9
```

Python Output

```
1 Valid YAML
2
```

Shell Script Solution

```
1 #!/bin/bash
2 yamllint config.yaml >/dev/null 2>&1
3
4 if [[ $? -eq 0 ]]; then
5     echo "Valid YAML"
6 else
7     echo "Invalid YAML"
8 fi
9
```

Shell Output

```
1 Valid YAML
2
```

34. Parse JSON and print a field

Python Solution

```
1 import json
2
```

```
3 data = json.loads('{ "name": "devops", "level": "senior"}')
4 print(data["name"])
5
```

Python Output

```
1 devops
2
```

Shell Script Solution

```
1 echo '{"name": "devops", "level": "senior"}' | jq -r '.name'
2
```

Shell Output

```
1 devops
2
```

35. Check disk space and alert if > 80%

Python Solution

```
1 import shutil
2
3 total, used, free = shutil.disk_usage("/")
4 usage = used / total * 100
5
6 if usage > 80:
7     print("ALERT: Disk usage high!", usage)
8 else:
9     print("Disk OK:", usage)
10
```

Python Output

```
1 Disk OK: 42.55
2
```

Shell Script Solution

```
1 usage=$(df / | awk 'NR==2 {print $5}' | tr -d '%')
2
3 if (( usage > 80 )); then
4     echo "ALERT: Disk usage high! $usage%"
5 else
6     echo "Disk OK: $usage%"
7 fi
8
```

Shell Output

```
1 Disk OK: 42%
2
```

36. Remove duplicate lines from a file

Python Solution

```
1 lines = open("input.txt").read().splitlines()
2 unique = list(dict.fromkeys(lines))
```

```
3 open("output.txt", "w").write("\n".join(unique))
4 print("Duplicates removed")
5
```

Python Output

```
1 Duplicates removed
2
```

Shell Script Solution

```
1 sort -u input.txt > output.txt
2 echo "Duplicates removed"
3
```

Shell Output

```
1 Duplicates removed
2
```

37. Find top 5 largest files in a directory

Python Solution

```
1 import os
2
3 path = "/var/log"
4 files = [(f, os.path.getsize(os.path.join(path, f))) for f in
5           os.listdir(path)]
6 top = sorted(files, key=lambda x: x[1], reverse=True)[:5]
7
8 for name, size in top:
9     print(name, size)
```

Python Output (example)

```
1 messages 204800
2 syslog 102400
3 kern.log 51200
4 apt.log 30000
5 dpkg.log 12000
6
```

Shell Script Solution

```
1 ls -Sh /var/log | head -5
2
```

Shell Output

```
1 messages
2 syslog
3 kern.log
4 apt.log
5 dpkg.log
6
```

38. Validate IPv4 address

Python Solution

```

1 import re
2
3 def valid_ip(ip):
4     pattern = r'^(\d{1,3}\.){3}\d{1,3}$'
5     if re.match(pattern, ip):
6         return all(0 <= int(x) <= 255 for x in ip.split('.'))
7     return False
8
9 print(valid_ip("10.10.10.256"))
10

```

Python Output

```

1 False
2

```

Shell Script Solution

```

1 ip="10.10.10.256"
2
3 if [[ $ip =~ ^([0-9]{1,3}\.){3}[0-9]{1,3}$ ]]; then
4     for part in ${ip//./ }; do
5         if (( part < 0 || part > 255 )); then
6             echo "Invalid IP"
7             exit 1
8         fi
9     done
10    echo "Valid IP"
11 else
12    echo "Invalid IP"
13 fi
14

```

Shell Output

```

1 Invalid IP
2

```

39. Generate a random password (12 chars)

Python Solution

```

1 import secrets
2 import string
3
4 chars = string.ascii_letters + string.digits
5 pwd = ''.join(secrets.choice(chars) for _ in range(12))
6 print(pwd)
7

```

Python Output (example)

```

1 aP8gF91LqXbC
2

```

Shell Script Solution

```

1 < /dev/urandom tr -dc 'A-Za-z0-9' | head -c12
2

```

Shell Output (example)

```

1 kA9dfT1HhQ8Z

```

40. Convert bytes to human-readable format

Python Solution

```
1 def readable(size):
2     for unit in ['B', 'KB', 'MB', 'GB', 'TB']:
3         if size < 1024:
4             return f"{size:.2f} {unit}"
5             size /= 1024
6
7 print(readable(123456789))
8
```

Python Output

```
1 117.74 MB
2
```

Shell Script Solution

```
1 num=123456789
2 echo $(numfmt --to=iec $num)
3
```

Shell Output

```
1 118M
2
```

41. Check if two files are identical (compare checksum)

Python Solution

```
1 import hashlib
2
3 def checksum(file):
4     h = hashlib.sha256()
5     with open(file, "rb") as f:
6         h.update(f.read())
7     return h.hexdigest()
8
9 if checksum("file1.txt") == checksum("file2.txt"):
10     print("Files are identical")
11 else:
12     print("Files differ")
13
```

Python Output

```
1 Files are identical
2
```

Shell Script Solution

```
1 c1=$(sha256sum file1.txt | awk '{print $1}')
2 c2=$(sha256sum file2.txt | awk '{print $1}')
3
4 if [[ "$c1" == "$c2" ]]; then
5     echo "Files are identical"
6
```

```
6 else
7     echo "Files differ"
8 fi
9
```

Shell Output

```
1 Files are identical
2
```

42. Extract all IP addresses from a log file

Python Solution

```
1 import re
2
3 log = open("access.log").read()
4 ips = re.findall(r'\b\d{1,3}(\.?\d{1,3}){3}\b', log)
5 print(set(ips))
6
```

Python Output (example)

```
1 {'10.0.0.1', '192.168.1.10'}
2
```

Shell Script Solution

```
1 grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' access.log | sort -u
2
```

Shell Output

```
1 10.0.0.1
2 192.168.1.10
3
```

43. Rotate a log file if size > 5MB

Python Solution

```
1 import os
2 size = os.path.getsize("app.log")
3
4 if size > 5 * 1024 * 1024:
5     os.rename("app.log", "app.log.bak")
6     print("Log rotated")
7 else:
8     print("Log OK")
9
```

Python Output

```
1 Log OK
2
```

Shell Script Solution

```
1 if [[ $(stat -c%s "app.log") -gt $((5*1024*1024)) ]]; then
2     mv app.log app.log.bak
3     echo "Log rotated"
4
```

```
4 else
5     echo "Log OK"
6 fi
7
```

Shell Output

```
1 Log OK
2
```

44. Find the count of running Docker containers

Python Solution

```
1 import subprocess
2
3 out = subprocess.check_output(["docker", "ps", "-q"]).decode().splitlines()
4 print("Running containers:", len(out))
5
```

Python Output

```
1 Running containers: 2
2
```

Shell Script Solution

```
1 count=$(docker ps -q | wc -l)
2 echo "Running containers: $count"
3
```

Shell Output

```
1 Running containers: 2
2
```

45. Convert a JSON file to a CSV file

Python Solution

```
1 import json
2 import csv
3
4 data = json.load(open("data.json"))
5
6 with open("output.csv", "w", newline="") as f:
7     writer = csv.DictWriter(f, fieldnames=data[0].keys())
8     writer.writeheader()
9     writer.writerows(data)
10
11 print("Converted to CSV")
12
```

Python Output

```
1 Converted to CSV
2
```

Shell Script Solution


```
1 jq -r '(.[] | keys_unsorted) as $keys |
2     $keys, map([. $keys[] ])[0] | @csv' data.json > output.csv
3
4 echo "Converted to CSV"
5
```

Shell Output

```
1 Converted to CSV
2
```

46. Check if Kubernetes pod is running (using Python)

Python Solution

```
1 import subprocess
2
3 pod="nginx"
4 cmd = subprocess.run(["kubectl", "get", "pod", pod, "-o", "jsonpath=
5     {.status.phase}"],
6                       text=True, capture_output=True)
7 print("Pod status:", cmd.stdout)
8
```

Python Output

```
1 Pod status: Running
2
```

Shell Script Solution

```
1 pod=nginx
2 status=$(kubectl get pod $pod -o jsonpath="{.status.phase}")
3 echo "Pod status: $status"
4
```

Shell Output

```
1 Pod status: Running
2
```

47. Print top CPU-consuming processes

Python Solution

```
1 import subprocess
2
3 out = subprocess.check_output(["ps", "-eo", "pid,comm,%cpu", "--sort=-
4     %cpu"])
5 print(out.decode().splitlines()[:10])
6
```

Python Output (example)

```
1 PID COMMAND %CPU
2 123 python 42.5
3 456 java 31.2
4 ...
5
```

Shell Script Solution

```
1 ps -eo pid,comm,%cpu --sort=-%cpu | head
2
```

Shell Output

```
1 PID COMMAND %CPU
2 123 python 42.5
3 456 java 31.2
4 ...
5
```

48. Create a backup of a directory with timestamp

Python Solution

```
1 import shutil
2 from datetime import datetime
3
4 ts = datetime.now().strftime("%Y%m%d%H%M%S")
5 shutil.make_archive(f"backup_{ts}", "zip", "/etc")
6 print("Backup created")
7
```

Python Output

```
1 Backup created
2
```

Shell Script Solution

```
1 ts=$(date +%Y%m%d%H%M%S)
2 zip -r "backup_${ts}.zip" /etc >/dev/null
3 echo "Backup created"
4
```

Shell Output

```
1 Backup created
2
```

49. Replace a string in a file (in-place)

Python Solution

```
1 text = open("app.conf").read()
2 new = text.replace("DEBUG", "INFO")
3 open("app.conf", "w").write(new)
4 print("Replaced")
5
```

Python Output

```
1 Replaced
2
```

Shell Script Solution

```
1 sed -i 's/DEBUG/INFO/g' app.conf
2 echo "Replaced"
```

```
3
```

Shell Output

```
1 Replaced
2
```

50. Extract failed login attempts from /var/log/auth.log

Python Solution

```
1 fails = []
2
3 with open("/var/log/auth.log") as f:
4     for line in f:
5         if "Failed password" in line:
6             fails.append(line.strip())
7
8 for f in fails:
9     print(f)
10
```

Python Output (example)

```
1 Jan 10 10:22:01 server sshd[2222]: Failed password for root
2
```

Shell Script Solution

```
1 grep "Failed password" /var/log/auth.log
2
```

Shell Output

```
1 Jan 10 10:22:01 server sshd[2222]: Failed password for root
2
```

51. Find all Python files larger than 1MB

Python Solution

```
1 import os
2
3 path = "/tmp"
4 for root, _, files in os.walk(path):
5     for f in files:
6         full = os.path.join(root, f)
7         if f.endswith(".py") and os.path.getsize(full) > 1 * 1024 *
8             1024:
9             print(full)
```

Python Output (example)

```
1 /tmp/test/script_large.py
2
```

Shell Script Solution

```
1 find /tmp -name "*.py" -size +1M
2
```

Shell Output

```
1 /tmp/test/script_large.py
2
```

52. Find which ports are listening

Python Solution

```
1 import subprocess
2
3 out = subprocess.check_output(["ss", "-tuln"]).decode()
4 print(out)
5
```

Python Output (example)

```
1 LISTEN 0 128 0.0.0.0:22 ...
2
```

Shell Script Solution

```
1 ss -tuln
2
```

Shell Output

```
1 LISTEN 0 128 0.0.0.0:22 ...
2
```

53. Convert a list of dictionaries to YAML

Python Solution

```
1 import yaml
2
3 data = [
4     {"name": "api", "port": 8080},
5     {"name": "web", "port": 80}
6 ]
7
8 print(yaml.dump(data))
9
```

Python Output

```
1 - name: api
2   port: 8080
3 - name: web
4   port: 80
5
```

Shell Script Solution

```
1 echo '["name":"api","port":8080},{"name":"web","port":80}]' | yq -P
2
```

Shell Output

```
1 - name: api
2   port: 8080
```

```
3 - name: web
4   port: 80
5
```

54. Get count of failed systemd services

Python Solution

```
1 import subprocess
2
3 out = subprocess.check_output(["systemctl", "state", "--
  failed"]).decode()
4 print(out)
5
```

Python Output (example)

```
1 0 loaded units listed.
2
```

Shell Script Solution

```
1 systemctl --failed
2
```

Shell Output

```
1 0 loaded units listed.
2
```

*55. Compress all .log files in a directory

Python Solution

```
1 import gzip
2 import os
3 import shutil
4
5 path = "/var/log"
6 for f in os.listdir(path):
7     if f.endswith(".log"):
8         with open(os.path.join(path, f), "rb") as src:
9             with gzip.open(os.path.join(path, f + ".gz"), "wb") as
  dest:
10                 shutil.copyfileobj(src, dest)
11                 print("Compressed:", f)
12
```

Python Output (example)

```
1 Compressed: syslog.log
2 Compressed: app.log
3
```

Shell Script Solution

```
1 gzip /var/log/*.log
2 echo "Compressed"
3
```

Shell Output

```
1 Compressed
2
```

56. Print current Kubernetes context

Python Solution

```
1 import subprocess
2
3 out = subprocess.check_output(["kubectl", "config", "current-
  context"]).decode()
4 print(out)
5
```

Python Output

```
1 gke_dev_cluster
2
```

Shell Script Solution

```
1 kubectl config current-context
2
```

Shell Output

```
1 gke_dev_cluster
2
```

57. Find and delete empty files

Python Solution

```
1 import os
2
3 path = "/tmp"
4 for root, _, files in os.walk(path):
5     for f in files:
6         full = os.path.join(root, f)
7         if os.path.getsize(full) == 0:
8             os.remove(full)
9             print("Deleted empty file:", full)
10
```

Python Output (example)

```
1 Deleted empty file: /tmp/test/a.txt
2
```

Shell Script Solution

```
1 find /tmp -type f -empty -delete -print
2
```

Shell Output

```
1 /tmp/test/a.txt
2
```

58. Monitor CPU usage every 2 seconds

Python Solution

```
1 import psutil
2 import time
3
4 for _ in range(3):
5     print("CPU:", psutil.cpu_percent(interval=2))
6
```

Python Output (example)

```
1 CPU: 12.5
2 CPU: 20.1
3 CPU: 14.0
4
```

Shell Script Solution

```
1 for i in {1..3}; do
2     mpstat 2 1 | grep "all" | awk '{print "CPU:", 100-$12}'
3 done
4
```

Shell Output (example)

```
1 CPU: 12.5
2 CPU: 20.1
3 CPU: 14.0
4
```

59. List mounted filesystems and available space

Python Solution

```
1 import shutil
2
3 usage = shutil.disk_usage("/")
4 print(f"Total: {usage.total}, Free: {usage.free}")
5
```

Python Output

```
1 Total: 120034123776, Free: 5641231232
2
```

Shell Script Solution

```
1 df -h /
2
```

Shell Output

```
1 Filesystem Size Used Avail Use% Mounted on
2 /dev/sda1 120G 65G 55G 55% /
3
```

60. Kill processes running on a specific port

Python Solution

```
1 import subprocess
2
3 port = "8080"
4 pids = subprocess.check_output(["lsof", "-t", f"-i:
{port}"]).decode().split()
5
6 for pid in pids:
7     subprocess.run(["kill", "-9", pid])
8
9 print("Killed processes on port", port)
10
```

Python Output

```
1 Killed processes on port 8080
2
```

Shell Script Solution

```
1 port=8080
2 kill -9 $(lsof -t -i:$port)
3 echo "Killed processes on port $port"
4
```

Shell Output

```
1 Killed processes on port 8080
2
```

61. Check if user exists on the system

Python Solution

```
1 import pwd
2
3 user = "ubuntu"
4
5 try:
6     pwd.getpwnam(user)
7     print("User exists")
8 except KeyError:
9     print("User does not exist")
10
```

Python Output

```
1 User exists
2
```

Shell Script Solution

```
1 user="ubuntu"
2
3 if id "$user" &>/dev/null; then
4     echo "User exists"
5 else
6     echo "User does not exist"
7 fi
8
```


Shell Output

```
1 User exists
2
```

*62. Count number of lines in all .log files

Python Solution

```
1 import glob
2
3 total = 0
4 for f in glob.glob("/var/log/*.log"):
5     total += sum(1 for _ in open(f))
6
7 print("Total log lines:", total)
8
```

Python Output

```
1 Total log lines: 15420
2
```

Shell Script Solution

```
1 wc -l /var/log/*.log | tail -1
2
```

Shell Output

```
1 15420 total
2
```

63. Send an HTTP GET request and print status code

Python Solution

```
1 import requests
2
3 r = requests.get("https://example.com")
4 print(r.status_code)
5
```

Python Output

```
1 200
2
```

Shell Script Solution

```
1 curl -o /dev/null -s -w "%{http_code}\n" https://example.com
2
```

Shell Output

```
1 200
2
```

64. Check if a directory is writable

Python Solution

```
1 import os
2
3 path = "/tmp"
4 print("Writable" if os.access(path, os.W_OK) else "Not writable")
5
```

Python Output

```
1 Writable
2
```

Shell Script Solution

```
1 path="/tmp"
2
3 if [[ -w "$path" ]]; then
4     echo "Writable"
5 else
6     echo "Not writable"
7 fi
8
```

Shell Output

```
1 Writable
2
```

65. Get list of all running AWS EC2 instances

Python Solution

```
1 import boto3
2
3 ec2 = boto3.client("ec2")
4 resp = ec2.describe_instances()
5
6 for r in resp["Reservations"]:
7     for i in r["Instances"]:
8         print(i["InstanceId"])
9
```

Python Output (example)

```
1 i-013ab2321abfa2bc1
2
```

Shell Script Solution

```
1 aws ec2 describe-instances --query
  "Reservations[].Instances[].InstanceId" --output text
2
```

Shell Output

```
1 i-013ab2321abfa2bc1
2
```

66. Track memory usage of a process by PID

Python Solution

```
1 import psutil
2
3 pid = 1
4 p = psutil.Process(pid)
5 print("Memory:", p.memory_info().rss)
6
```

Python Output

```
1 Memory: 34570240
2
```

Shell Script Solution

```
1 pid=1
2 ps -o rss= -p $pid
3
```

Shell Output

```
1 34570
2
```

67. Find all symlinks in a directory

Python Solution

```
1 import os
2
3 path = "/usr/bin"
4 for f in os.listdir(path):
5     full = os.path.join(path, f)
6     if os.path.islink(full):
7         print(full)
8
```

Python Output (example)

```
1 /usr/bin/python
2
```

Shell Script Solution

```
1 find /usr/bin -type l
2
```

Shell Output

```
1 /usr/bin/python
2
```

68. Remove files older than 30 days

Python Solution

```
1 import os, time
2
```

```

3 path = "/tmp"
4 now = time.time()
5
6 for root, _, files in os.walk(path):
7     for f in files:
8         full = os.path.join(root, f)
9         if now - os.path.getmtime(full) > 30 * 86400:
10             os.remove(full)
11             print("Deleted:", full)
12

```

Python Output

```

1 Deleted: /tmp/oldfile.txt
2

```

Shell Script Solution

```

1 find /tmp -type f -mtime +30 -print -delete
2

```

Shell Output

```

1 /tmp/oldfile.txt
2

```

69. Ping a list of servers and print reachable/unreachable

Python Solution

```

1 import os
2
3 servers = ["google.com", "localhost", "invalid.host"]
4
5 for s in servers:
6     status = os.system(f"ping -c1 -W1 {s} > /dev/null 2>&1")
7     print(s, "reachable" if status == 0 else "unreachable")
8

```

Python Output

```

1 google.com reachable
2 localhost reachable
3 invalid.host unreachable
4

```

Shell Script Solution

```

1 servers=("google.com" "localhost" "invalid.host")
2
3 for s in "${servers[@]}; do
4     if ping -c1 -W1 "$s" >/dev/null 2>&1; then
5         echo "$s reachable"
6     else
7         echo "$s unreachable"
8     fi
9 done
10

```

Shell Output

```

1 google.com reachable
2 localhost reachable
3 invalid.host unreachable

```

70. Print CPU core count

Python Solution

```
1 import os
2 print(os.cpu_count())
3
```

Python Output

```
1 8
2
```

Shell Script Solution

```
1 nproc
2
```

Shell Output

```
1 8
2
```

71. Get Docker image size

Python Solution

```
1 import subprocess, json
2
3 image = "nginx:latest"
4 out = subprocess.check_output(["docker", "image", "inspect",
5 image]).decode()
6 info = json.loads(out)[0]
7 print("Size:", info["Size"])
8
```

Python Output

```
1 Size: 142345678
2
```

Shell Script Solution

```
1 docker image inspect nginx:latest --format '{{.Size}}'
2
```

Shell Output

```
1 142345678
2
```

72. Check SSL certificate expiry date

Python Solution

```
1 import ssl, socket
2 from datetime import datetime
```

```

3
4 host = "google.com"
5 ctx = ssl.create_default_context()
6
7 with socket.create_connection((host, 443)) as sock:
8     with ctx.wrap_socket(sock, server_hostname=host) as ss:
9         cert = ss.getpeercert()
10
11 expiry = cert['notAfter']
12 print("Expiry:", expiry)
13

```

Python Output

```

1 Expiry: Nov  5 12:00:00 2025 GMT
2

```

Shell Script Solution

```

1 echo | openssl s_client -connect google.com:443 2>/dev/null \
2     | openssl x509 -noout -enddate
3

```

Shell Output

```

1 notAfter=Nov  5 12:00:00 2025 GMT
2

```

73. Generate a random secure password

Python Solution

```

1 import secrets
2 import string
3
4 chars = string.ascii_letters + string.digits + "!@#$$%"
5 pwd = ''.join(secrets.choice(chars) for _ in range(12))
6 print(pwd)
7

```

Python Output (example)

```

1 A9dk!pQ3wR2@
2

```

Shell Script Solution

```

1 tr -dc 'A-Za-z0-9!@#$$%' < /dev/urandom | head -c 12
2 echo
3

```

Shell Output

```

1 Gs7@AdkP99#Q
2

```

74. Check if Kubernetes deployment is fully rolled out

Python Solution

```

1 import subprocess

```

```
2
3 deploy = "nginx"
4 out = subprocess.run(
5     ["kubectl", "rollout", "status", f"deploy/{deploy}"],
6     capture_output=True, text=True
7 )
8
9 print(out.stdout)
10
```

Python Output

```
1 deployment "nginx" successfully rolled out
2
```

Shell Script Solution

```
1 kubectl rollout status deploy/nginx
2
```

Shell Output

```
1 deployment "nginx" successfully rolled out
2
```

75. Compare two JSON files for differences

Python Solution

```
1 import json
2
3 a = json.load(open("a.json"))
4 b = json.load(open("b.json"))
5
6 if a == b:
7     print("Identical")
8 else:
9     print("Different")
10
```

Python Output

```
1 Different
2
```

Shell Script Solution

```
1 diff <(jq -S . a.json) <(jq -S . b.json)
2
```

Shell Output

```
1 2c2
2 <  "version": 1
3 ---
4 >  "version": 2
5
```

76. Check network latency to a server

Python Solution

```
1 import subprocess
2
3 host = "8.8.8.8"
4 out = subprocess.check_output(["ping", "-c1", host]).decode()
5 print(out.split("time=")[1].split()[0])
6
```

Python Output

```
1 12.3 ms
2
```

Shell Script Solution

```
1 ping -c1 8.8.8.8 | grep 'time=' | awk -F'time=' '{print $2}'
2
```

Shell Output

```
1 12.3 ms
2
```

77. Read last 50 lines of a file

Python Solution

```
1 from collections import deque
2
3 with open("app.log") as f:
4     last = deque(f, 50)
5
6 for line in last:
7     print(line, end="")
8
```

Python Output

```
1 (last 50 lines printed)
2
```

Shell Script Solution

```
1 tail -50 app.log
2
```

Shell Output

```
1 (last 50 lines printed)
2
```

*78. Create a zip archive of all .conf files

Python Solution

```
1 import zipfile
2 import glob
3
4 with zipfile.ZipFile("configs.zip", "w") as z:
5     for f in glob.glob("/etc/*.conf"):
6         z.write(f)
7
```



```
8 print("Archive created")
9
```

Python Output

```
1 Archive created
2
```

Shell Script Solution

```
1 zip configs.zip /etc/*.conf >/dev/null
2 echo "Archive created"
3
```

Shell Output

```
1 Archive created
2
```

79. Find duplicate files based on hash

Python Solution

```
1 import hashlib, os
2
3 hashes = {}
4 dup = []
5
6 path = "/tmp"
7 for root, _, files in os.walk(path):
8     for f in files:
9         full = os.path.join(root, f)
10        h = hashlib.md5(open(full, "rb").read()).hexdigest()
11        if h in hashes:
12            dup.append(full)
13        else:
14            hashes[h] = full
15
16 print("Duplicates:", dup)
17
```

Python Output

```
1 Duplicates: ['/tmp/data/file2.txt']
2
```

Shell Script Solution

```
1 find /tmp -type f -exec md5sum {} \; | sort | uniq -w32 -D
2
```

Shell Output

```
1 a7cb12ef89d file1.txt
2 a7cb12ef89d file2.txt
3
```

80. Print system uptime

Python Solution

```
1 import subprocess
2
3 out = subprocess.check_output(["uptime", "-p"]).decode()
4 print(out)
5
```

Python Output

```
1 up 3 days, 2 hours
2
```

Shell Script Solution

```
1 uptime -p
2
```

Shell Output

```
1 up 3 days, 2 hours
2
```

81 — Monitor a log file for a regex and alert once seen

Python

```
1 import re, time
2
3 pattern = re.compile(r'ERROR.*database')
4 found = False
5
6 with open("app.log") as f:
7     # start at end
8     f.seek(0, 2)
9     for _ in range(100): # check up to 100 iterations for demo
10         line = f.readline()
11         if not line:
12             time.sleep(0.1)
13             continue
14         if pattern.search(line):
15             print("ALERT: database error detected")
16             found = True
17             break
18
19 if not found:
20     print("No match yet")
21
```

Bash

```
1 #!/bin/bash
2 tail -n0 -F app.log 2>/dev/null | \
3 while read -r line; do
4     if [[ $line =~ ERROR.*database ]]; then
5         echo "ALERT: database error detected"
6         pkill -P $$ tail # stop tail in demo
7         break
8     fi
9 done
10
```

Expected output

```
1 ALERT: database error detected
2
```

82 — Create a small HTTP health-check server (returns 200/500)

Python

```
1 from http.server import BaseHTTPRequestHandler, HTTPServer
2
3 class H(BaseHTTPRequestHandler):
4     def do_GET(self):
5         # simple health-check that returns 200
6         if self.path == "/health":
7             self.send_response(200)
8             self.end_headers()
9             self.wfile.write(b"OK")
10        else:
11            self.send_response(404)
12            self.end_headers()
13
14 if __name__ == "__main__":
15     srv = HTTPServer(("0.0.0.0", 8000), H)
16     print("Serving on :8000")
17     try:
18         srv.serve_forever()
19     except KeyboardInterrupt:
20         srv.server_close()
21
```

Bash (using netcat for a one-off)

```
1 #!/bin/bash
2 # simple one-shot health-response on port 8000 (netcat required)
3 while true; do
4     { echo -e "HTTP/1.1 200 OK\r\nContent-Length: 2\r\n\r\nOK"; } | nc -l
5     -p 8000 -q 1
6 done
```

Expected output (server stdout)

```
1 Serving on :8000
2
```

(Hitting `GET /health` returns body `OK` with HTTP 200.)

83 — Atomic write: write temp file then rename

Python

```
1 import os
2 import tempfile
3
4 def atomic_write(path, data):
5     dirn = os.path.dirname(path) or "."
6     fd, tmp = tempfile.mkstemp(dir=dirn)
7     try:
8         with os.fdopen(fd, "w") as f:
9             f.write(data)
10        os.replace(tmp, path)
11    finally:
12        if os.path.exists(tmp):
13            os.remove(tmp)
14
15 atomic_write("config.json", '{"key":"value"}')
16 print("Wrote atomically")
17
```

Bash

```
1 #!/bin/bash
```

```

2 path="config.json"
3 tmp="{path}.$$"
4 echo '{"key":"value"}' > "$tmp" && mv -f "$tmp" "$path" && echo "Wrote
  atomically"
5

```

Expected output

```

1 Wrote atomically
2

```

84 — Exponential backoff HTTP retry (idempotent GET)

Python

```

1 import time, requests
2
3 def get_with_backoff(url, retries=4):
4     for attempt in range(retries):
5         try:
6             r = requests.get(url, timeout=2)
7             r.raise_for_status()
8             return r.text
9         except Exception as e:
10            if attempt == retries - 1:
11                raise
12            time.sleep(0.5 * (2 ** attempt))
13
14 print("Status length:", len(get_with_backoff("https://example.com")))
15

```

Bash

```

1 #!/bin/bash
2 url="https://example.com"
3 retries=4
4 for i in $(seq 0 $((retries-1))); do
5     if curl -fsS "$url" -o /dev/null; then
6         echo "reachable"
7         break
8     fi
9     if [ "$i" -eq $((retries-1)) ]; then
10        echo "failed"
11        break
12    fi
13    sleep $((1 << i))
14 done
15

```

Expected output

```

1 Status length: 1256
2

```

(or `reachable` for the bash script)

85 — Create a small CLI that reads key=value pairs from stdin and prints as JSON

Python

```

1 import sys, json
2
3 data = {}
4 for line in sys.stdin:
5     line = line.strip()
6     if not line:

```

```

7         continue
8     if "=" in line:
9         k,v = line.split("=",1)
10        data[k] = v
11
12 print(json.dumps(data))
13

```

Bash

```

1  #!/bin/bash
2  declare -A kv
3  while IFS='=' read -r k v; do
4      kv["$k"]="$v"
5  done
6  # print as simple JSON
7  printf "{"
8  first=1
9  for k in "${!kv[@]}"; do
10     if [ $first -eq 1 ]; then first=0; else printf ","; fi
11     printf "\"%s\": \"%s\" " "$k" "${kv[$k]}"
12 done
13 printf "}\n"
14

```

Example stdin

```

1 name=web
2 port=8080
3

```

Expected output

```

1 {"name":"web","port":"8080"}
2

```

86 — Generate a short unique ID (URL-safe)

Python

```

1 import secrets, base64
2
3 def short_id(n=9):
4     return
5     base64.urlsafe_b64encode(secrets.token_bytes(n)).rstrip(b'=').decode()
6
7 print(short_id())

```

Bash

```

1 #!/bin/bash
2 head -c9 /dev/urandom | base64 | tr -d '=' | tr '/\n' 'a\n' | head -
3 c12
4 echo

```

Expected output (example)

```

1 Xk3f9AbtQz4
2

```

(will vary each run)

87 — Lookup A record for a domain (DNS query)

Python

```
1 import socket
2
3 def a_records(domain):
4     try:
5         return socket.getaddrinfo(domain, None, family=socket.AF_INET)
6     except Exception:
7         return []
8
9 recs = a_records("example.com")
10 ips = sorted({r[4][0] for r in recs})
11 print(ips)
12
```

Bash

```
1 #!/bin/bash
2 dig +short A example.com | sort -u
3
```

Expected output

```
1 ['93.184.216.34']
2
```

(or 93.184.216.34 from dig)

88 — Simple circuit-breaker: stop trying after N failures for T seconds

Python

```
1 import time
2
3 class Circuit:
4     def __init__(self, max_fail=3, cooldown=10):
5         self.fail = 0
6         self.max_fail = max_fail
7         self.cooldown = cooldown
8         self.open_until = 0
9
10    def call(self, fn, *a, **k):
11        now = time.time()
12        if now < self.open_until:
13            raise RuntimeError("circuit open")
14        try:
15            res = fn(*a, **k)
16            self.fail = 0
17            return res
18        except Exception:
19            self.fail += 1
20            if self.fail >= self.max_fail:
21                self.open_until = now + self.cooldown
22            raise
23
24 # demo
25 c = Circuit(max_fail=2, cooldown=1)
26 def flaky():
27     raise RuntimeError("fail")
28
29 try:
30     c.call(flaky)
31 except: print("attempt1 failed")
32 try:
33     c.call(flaky)
34 except: print("attempt2 failed -> circuit opens")
```

```

35 try:
36     c.call(flaky)
37 except Exception as e:
38     print("circuit state:", e)
39

```

Bash (simple state with file)

```

1  #!/bin/bash
2  statefile="/tmp/circuit.state"
3  max_fail=2
4  cooldown=10
5
6  failcount=0
7  open_until=0
8  if [ -f "$statefile" ]; then
9      read failcount open_until < "$statefile"
10 fi
11
12 now=$(date +%s)
13 if [ "$now" -lt "$open_until" ]; then
14     echo "circuit open"
15     exit 1
16 fi
17
18 # simulate failure
19 if true; then
20     failcount=$((failcount+1))
21     if [ "$failcount" -ge "$max_fail" ]; then
22         open_until=$((now+cooldown))
23         echo "$failcount $open_until" > "$statefile"
24         echo "circuit opened"
25         exit 1
26     fi
27 fi
28 echo "$failcount $open_until" > "$statefile"
29 echo "attempt recorded"
30

```

Expected output

```

1  attempt1 failed
2  attempt2 failed -> circuit opens
3  circuit state: circuit open
4

```

(or `circuit opened` for bash demo)

89 — Tail multiple files concurrently (non-blocking)

Python

```

1  import selectors, sys
2
3  sel = selectors.DefaultSelector()
4
5  def register(f):
6      sel.register(f, selectors.EVENT_READ)
7
8  files = [open("a.log"), open("b.log")]
9  for f in files:
10     register(f)
11
12 for _ in range(100):
13     for key, _ in sel.select(timeout=0.1):
14         line = key.fileobj.readline()
15         if line:
16             print(key.fileobj.name, line.rstrip())

```

Bash

```

1 #!/bin/bash
2 # uses multitail if available; fallback to background tails
3 if command -v multitail >/dev/null; then
4     multitail a.log b.log
5 else
6     tail -F a.log & tail -F b.log &
7     wait
8 fi
9

```

Expected output

```

1 a.log new line 1
2 b.log another line
3 a.log new line 2
4

```

90 — Verify a GPG signature of a file

Python (shell out to gpg)

```

1 import subprocess
2
3 ret = subprocess.run(["gpg", "--verify", "file.sig", "file"],
4 capture_output=True, text=True)
5 print(ret.returncode, ret.stderr.splitlines()[-1] if ret.stderr else
6 ret.stdout)
7

```

Bash

```

1 gpg --verify file.sig file && echo "Signature OK" || echo "Signature
2 invalid"
3

```

Expected output

```

1 Signature OK
2

```

(or `Signature invalid` if verification fails)

91 — Get CPU, Memory, and Disk usage in one output

Python

```

1 import psutil
2
3 print("CPU %:", psutil.cpu_percent())
4 print("Memory %:", psutil.virtual_memory().percent)
5 print("Disk %:", psutil.disk_usage('/').percent)
6

```

Shell

```

1 echo "CPU %: $(top -bn1 | grep Cpu | awk '{print $2}')"
2 echo "Memory %: $(free | awk 'Mem:/ {printf(\"%.2f\", $3/$2 * 100)}')"
3 echo "Disk %: $(df -h / | awk 'NR==2{print $5}')"
4

```


Output (example)

```
1 CPU %: 12.6
2 Memory %: 47.3
3 Disk %: 68%
4
```

92 — Validate YAML file syntax

Python

```
1 import yaml
2
3 try:
4     yaml.safe_load(open("deployment.yaml"))
5     print("Valid YAML")
6 except Exception as e:
7     print("Invalid YAML:", e)
8
```

Shell

```
1 yamllint deployment.yaml && echo "Valid YAML"
2
```

Output

```
1 Valid YAML
2
```

93 — Generate a random password with special chars

Python

```
1 import secrets, string
2
3 chars = string.ascii_letters + string.digits + "!@#$%^&*\"
4 password = ''.join(secrets.choice(chars) for _ in range(16))
5 print(password)
6
```

Shell

```
1 tr -dc 'A-Za-z0-9!@#$%^&*\" </dev/urandom | head -c16; echo
2
```

Output

```
1 L8@kFa19#PqT77Ae
2
```

94 — List all open network listening ports

Python

```
1 import subprocess
2
3 output = subprocess.check_output(["ss", "-tulpn"]).decode()
4 print(output)
5
```

Shell

```
1 ss -tulpn
2
```

Output

```
1 LISTEN 0 128 0.0.0.0:22 ...
2 LISTEN 0 128 :::80 ...
3
```

95 — Compress a directory and print size before/after

Python

```
1 import os, tarfile
2
3 def size(p):
4     return sum(os.path.getsize(os.path.join(r,f))
5                 for r,_,fs in os.walk(p) for f in fs)
6
7 print("Before:", size("data"), "bytes")
8
9 with tarfile.open("data.tar.gz", "w:gz") as tar:
10     tar.add("data", arcname="data")
11
12 print("After:", os.path.getsize("data.tar.gz"), "bytes")
13
```

Shell

```
1 echo "Before: $(du -sb data | awk '{print $1}') bytes"
2 tar -czf data.tar.gz data
3 echo "After: $(stat -c%s data.tar.gz) bytes"
4
```

Output

```
1 Before: 42344 bytes
2 After: 7821 bytes
3
```

96 — Create a retry wrapper for any command

Python

```
1 import subprocess, time
2
3 def retry(cmd, attempts=3, wait=2):
4     for i in range(attempts):
5         if subprocess.call(cmd, shell=True) == 0:
6             print("Success")
7             return
8         print("Retrying...")
9         time.sleep(wait)
10    print("Failed")
11
12 retry("curl -I https://example.com")
13
```

Shell

```
1 cmd="curl -I https://example.com"
2 for i in {1..3}; do
3     $cmd && { echo "Success"; break; }
```

```
4     echo "Retrying..."
5     sleep 2
6 done
7
```

Output

```
1 Success
2
```

97 — Create a file watcher (trigger action if updated)

Python

```
1 import os, time
2
3 path = "config.yaml"
4 last = os.path.getmtime(path)
5
6 while True:
7     time.sleep(1)
8     now = os.path.getmtime(path)
9     if now != last:
10         print("File updated")
11         last = now
12         break
13
```

Shell

```
1 inotifywait -e modify config.yaml && echo "File updated"
2
```

Output

```
1 File updated
2
```

98 — Convert CSV to JSON

Python

```
1 import csv, json
2
3 with open("data.csv") as f:
4     reader = csv.DictReader(f)
5     print(json.dumps(list(reader), indent=2))
6
```

Shell

```
1 csvjson data.csv
2
```

Output

```
1 [
2   {"name": "app", "port": "8080"},
3   {"name": "db", "port": "5432"}
4 ]
5
```

99 — Parse and validate a cron expression (simple checks)

Python

```
1 import re
2
3 cron = "0 2 * * *"
4 pattern = r'^(\*[0-59]+) (\*[0-23]+) (\*[0-31]+) (\*[0-12]+) (\*[0-7]+)$'
5
6 if re.match(pattern, cron):
7     print("Valid cron")
8 else:
9     print("Invalid cron")
10
```

Shell

```
1 cron="0 2 * * *"
2 echo "$cron" | grep -Eq '^(\[0-59\]+) (\[0-23\]+) (\[0-31\]+) (\[0-12\]+) (\[0-7\]+)$' \
3   && echo "Valid cron" || echo "Invalid cron"
4
```

Output

```
1 Valid cron
2
```

100 — Build a simple CLI menu in Python + shell

Python

```
1 while True:
2     print("1) Show date")
3     print("2) Show hostname")
4     print("3) Exit")
5     choice = input("Choose: ")
6
7     if choice == "1":
8         import datetime
9         print(datetime.datetime.now())
10    elif choice == "2":
11        import socket
12        print(socket.gethostname())
13    elif choice == "3":
14        print("Bye")
15        break
16    else:
17        print("Invalid")
18
```

Shell

```
1 while true; do
2     echo "1) Show date"
3     echo "2) Show hostname"
4     echo "3) Exit"
5     read -p "Choose: " choice
6     case $choice in
7         1) date ;;
8         2) hostname ;;
9         3) echo "Bye"; exit ;;
10        *) echo "Invalid" ;;
11    esac
12 done
13
```

Output (sample)

```
1 1) Show date
2 2) Show hostname
3 3) Exit
4 Choose: 1
5 2025-02-21 14:03:31
6
```
